```
  SQS.sendMessage({'processId': processId, 'action': 'process'});
}
```

If you retry now, you'll make multiple updates to Dynamo, which doesn't hurt. And you may send multiple SQS messages, which also doesn't hurt, because the SQS message consumer must be idempotent as well. The same applies to the transition `Uploaded → Processed`.

One little thing is still missing. So far, the code will fetch an item from DynamoDB and will update the item a few lines after that. In between, another process might have set the state to `Uploaded` already. Luckily, the database supports conditional updates, which allows us to reduce all the logic into a single DynamoDB request. DynamoDB will evaluate the condition before updating the item, as shown here (pseudocode):

```
uploaded(s3Key) {
  process = DynamoDB.getItem(processId)
  DynamoDB.updateItem(processId, {
    'state': 'Uploaded',
    'rawS3Key': s3Key,
    condition: 'NOT state IN(Created, Uploaded)'
  })
  SQS.sendMessage({'processId': processId, 'action': 'process'});
}
```

Next, you'll begin to implement the Imagery server.

## *16.3.2 Implementing a fault-tolerant web service*

We'll split the Imagery application into two parts: the web servers and the workers. As illustrated in figure 16.11, the web servers provide the